# Project 1 Part 2

**Important:**

- Write your name as well as your NU ID on your assignment. Please number your problems.
- Submit both results and your code.
- Give complete answers. Do not just give the final answer; instead show steps you went through to get there and explain what you are doing. Do not leave out critical intermediate steps.
- This assignment must be submitted electronically through Gradescope by April 24th 2026 (Friday) by 11:59 PM.
- All of your codes must be commented.
- All of your plots must be labeled, titled and have a legend (when applicable).

This assignment serves as the first part of a larger project, focused on healthcare, that you will be completing in this class. The project consists of developing a (simplistic) machine learning model that allows you to predict the diastolic blood pressure and systolic blood pressure of a patient given the total cholesterol level in their blood. In this part of the project, you will explore the gradient descent algorithm and start implementing the framework needed to build the predictive model.

The main goals of this assignment are the following:

- Derive cost function for model optimization.
- Implement gradient function to the cost function.
- Evaluate the performance of your model on different data sets.

## 1 Cost Function to Evaluate the Performance of a Predictive Model

Suppose you are given $n$ data points of the form $\{(x^{(i)}, y^{(i)}), i = 1, 2, ..., n\}$, where $x^{(i)}$ is the total cholesterol level in patient $i$'s blood and $y^{(i)}$ is the diastolic blood pressure of patient $i$. Your current goal is to come up with a model that predicts "well" the diastolic blood pressure of any patient given their total blood cholesterol level. The data considered is shown in figure 1 and uploaded on Canvas and uploaded as *data_chol_dias_pressure.txt*. Note that the current data set is a synthetic data set that I created and not an actual medical data set.

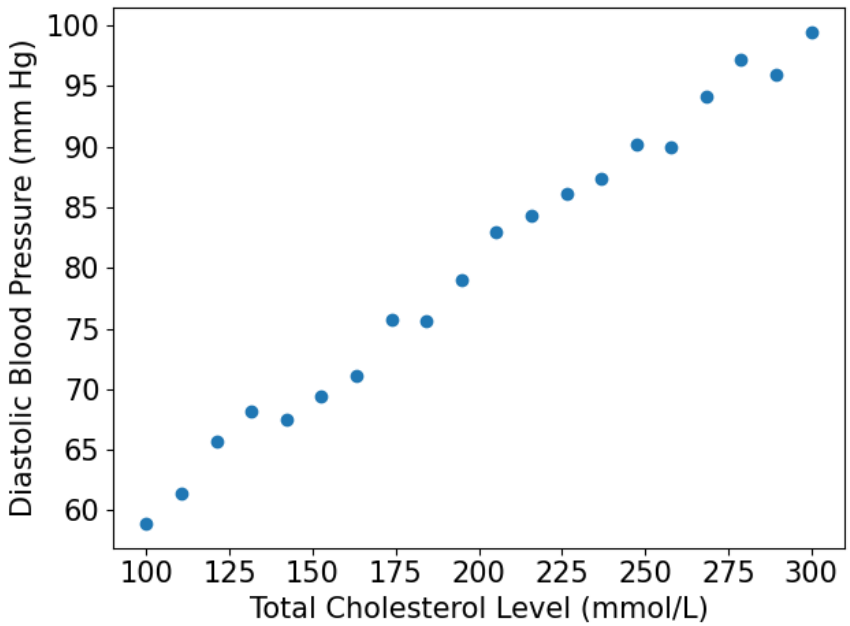


Figure 1: Plot illustrating the diastolic blood pressure of different patients given their total blood cholesterol level. Every patient is represented by a blue dot in the plot.

The data shown in figure 1 looks generally linear. One possible option would be to find a linear function $f(x) = ax + b$ for which the line would be as close as possible to blue dots.
That is, our goal is to find some optimal values of the slope $a$ and intercept $b$ for which,

$$f(x^{(i)}) = ax^{(i)} + b \approx y^{(i)}, \tag{1}$$

for all $n$ patients. We need to compute the magnitude of the difference of $ax^{(i)} + b$ and $y^{(i)}$ for every patient $i$ and try to minimize these quantities. The magnitude has to be a non negative number. One option would be to consider the square of the difference of $ax^{(i)} + b$ and $y^{(i)}$ for every patient $i$ to result in non-negative values. This expression would be,

$$\left(ax^{(i)} + b - y^{(i)}\right)^2. \tag{2}$$

Additionally, we seek to consider the above expression for every patient $i$ to find the optimal line that is as close as possible to all the blue dots shown in figure 1. This is why, we will analyze the cost function,

$$g(a, b) = \left(ax^{(1)} + b - y^{(1)}\right)^2 + \left(ax^{(2)} + b - y^{(2)}\right)^2 + ... + \left(ax^{(n)} + b - y^{(n)}\right)^2, \tag{3}$$

to take into account all patients.

Our goal is to find the optimal $a$ and $b$ which minimize $g(a, b)$ so that the optimal line given by $y = ax + b$ is as close as possible to all the data points.

**1.** Read the data given on Canvas and split into two arrays $x$ and $y$. The first column of the data should be stored in $x$ and the second column of the data should be stored in $y$. Each array $x$ and $y$ will have 20 elements has the data set given to you consists of 20 patients. You can do that using the following code:

```
import numpy as np

def load_data(filename):
    '''
    Load the data into two arrays
```

```
    Args:
        filename: string representing the name of the file
            containing the data (x,y)
    Return:
        array containing the values of x
        array containing the values of y

    '''
    data = np.loadtxt(filename)
    return data[:, 0], data[:, 1]

x,y = load_data('data_chol_dias_pressure.txt')
```

2. Write a function which computes $g(a, b)$ for any $a$ and $b$. This function should take as input $a$, $b$, $x$ and $y$.

3. Notice that $g(a, b)$ is a function of two variables. Apply the gradient descent algorithm developed in the second part of the project, for functions of two variables, to compute the optimal values of $a$ and $b$, denoted by, $a^*$ and $b^*$, respectively.

4. Plot the line $y = a^*x + b^*$ along with the data points shown in figure 1. How does the plotted line perform?

5. Was it easy to find the correct values for the initial parameters for gradient descent to converge? Why do you think this is happening?
   **Hint:** Scaling your dataset, i.e. subtracting from every data the mean of the dataset and dividing by the standard deviation of the dataset will help.

## 2 Non-linear Data

You were just informed that the blood pressure monitor machine used in the previous section had manufacturing errors and resulted in incorrect data. You now used a new blood pressure monitor machine that is less prone to errors and obtained the data shown in figure 2. This data is stored in *data_chol_dias_pressure_non_lin.txt* and uploaded on Canvas.

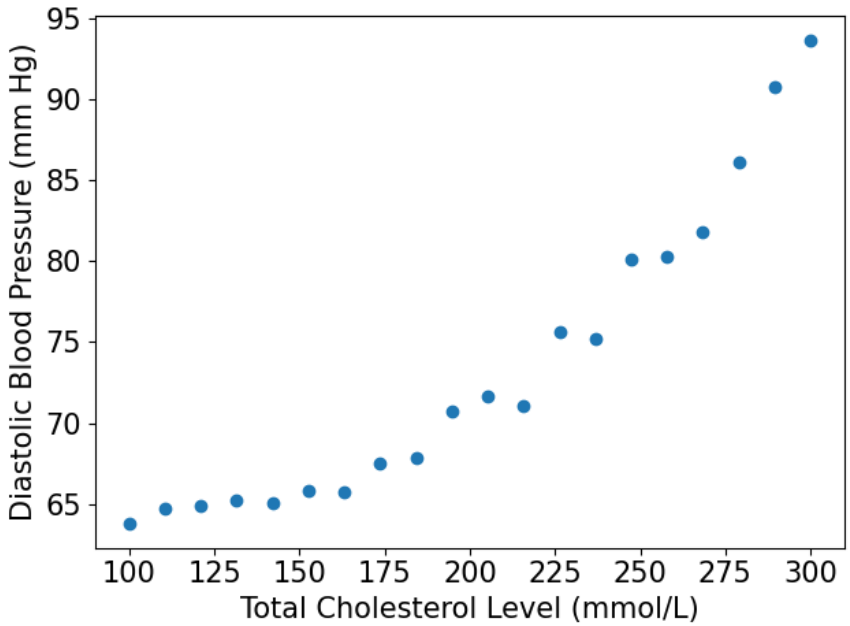


Figure 2: Plot illustrating the diastolic blood pressure of different patients given their total blood cholesterol level, using the new blood pressure monitor machine. Every patient is represented by a blue dot in the plot.

1. Read the data given on Canvas and split into two arrays $x$ and $y$. The first column of the data should be stored in $x$ and the second column of the data should be stored in $y$. Each array $x$ and $y$ will have 20 elements has the data set given to you consists of 20 patients.

2. Apply the gradient descent algorithm developed in the second part of the project, for functions of two variables, on $g(a, b)$, computed according to the new data, to compute the optimal values of $a$ and $b$, denoted by, $a^*$ and $b^*$, respectively.

3. Plot the line $y = a^*x + b^*$ along with the data points shown in figure 2. How does the plotted line perform now? Why is the optimal line performing different for the updated data set?

4. What possible modifications can you implement to your model to improve its performance? There isn't one correct answer here, what matters to me is your reasoning. Think whether the function $f(x)$ adopted is suitable for the new data set. Implement your proposed change(s).

# 3 Report

Write a report that explains how you designed your code to implement gradient descent for functions of several variables on the cost function $g(a, b)$. Summarize what you learnt about how the choice of $f(x)$ can affect the performance on the model depending on the data set.